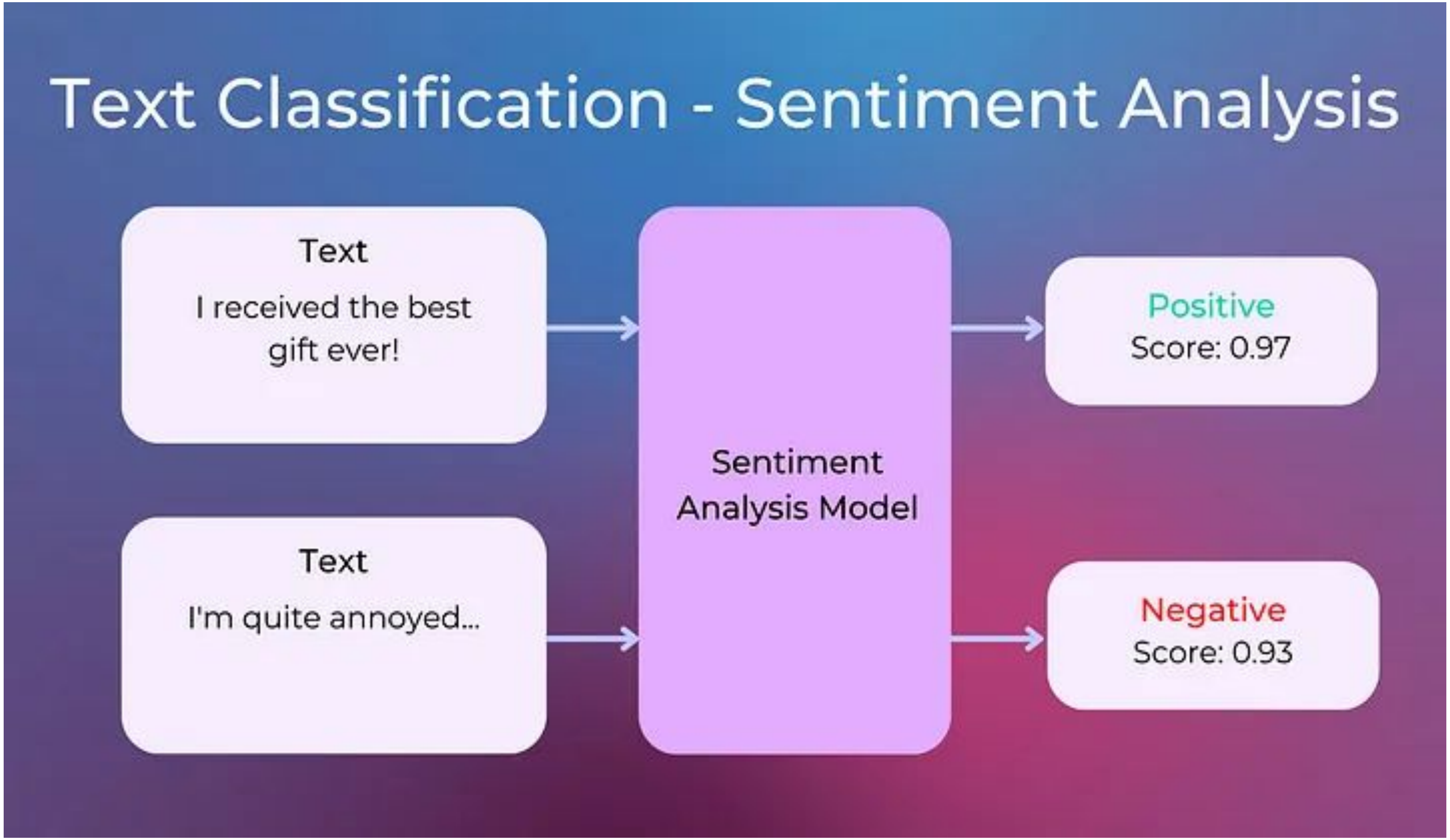
Overview of NLP tasks

NLP tasks around us:

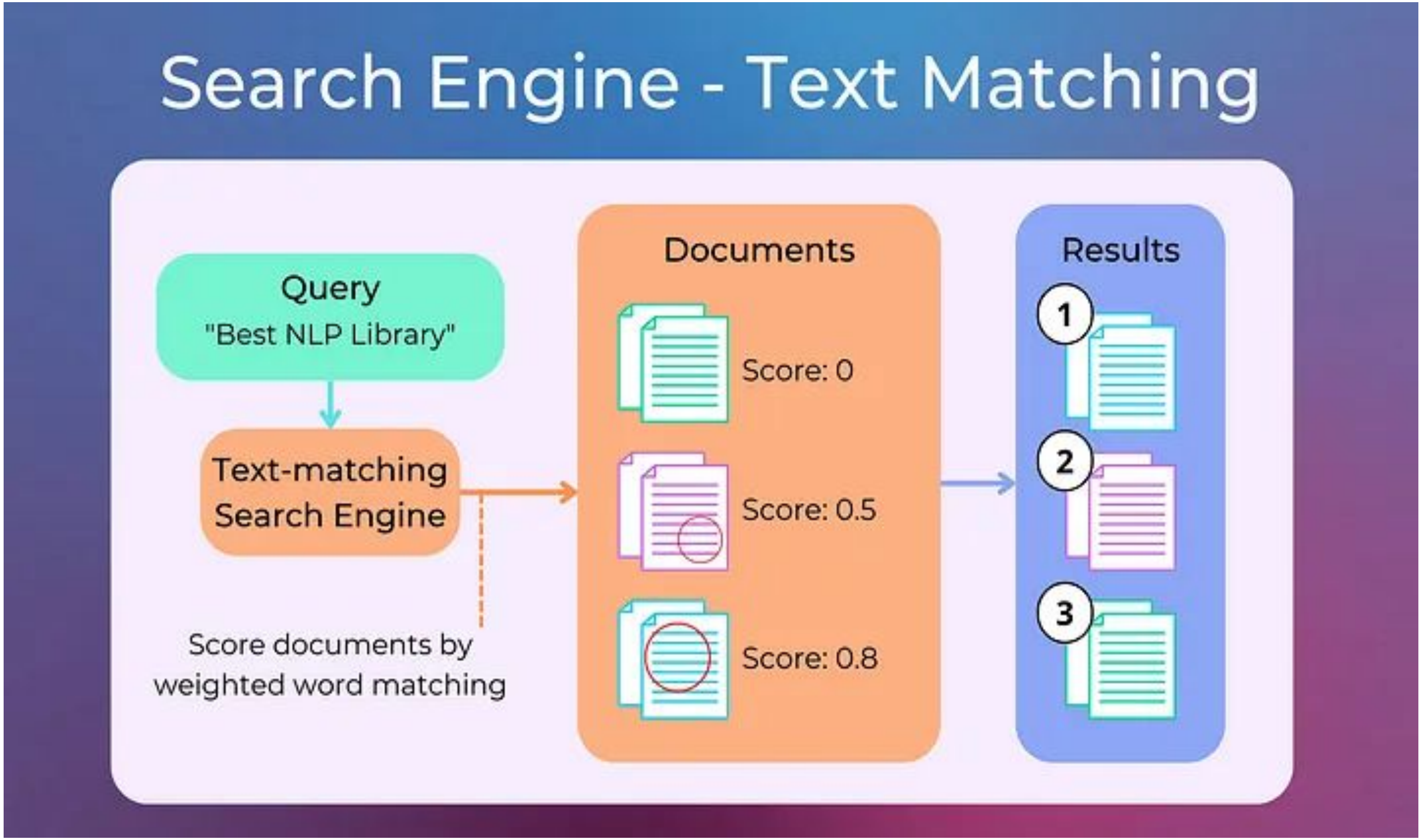
— suggest in search
— automatic gmail replies
— machine translation

Chatbot

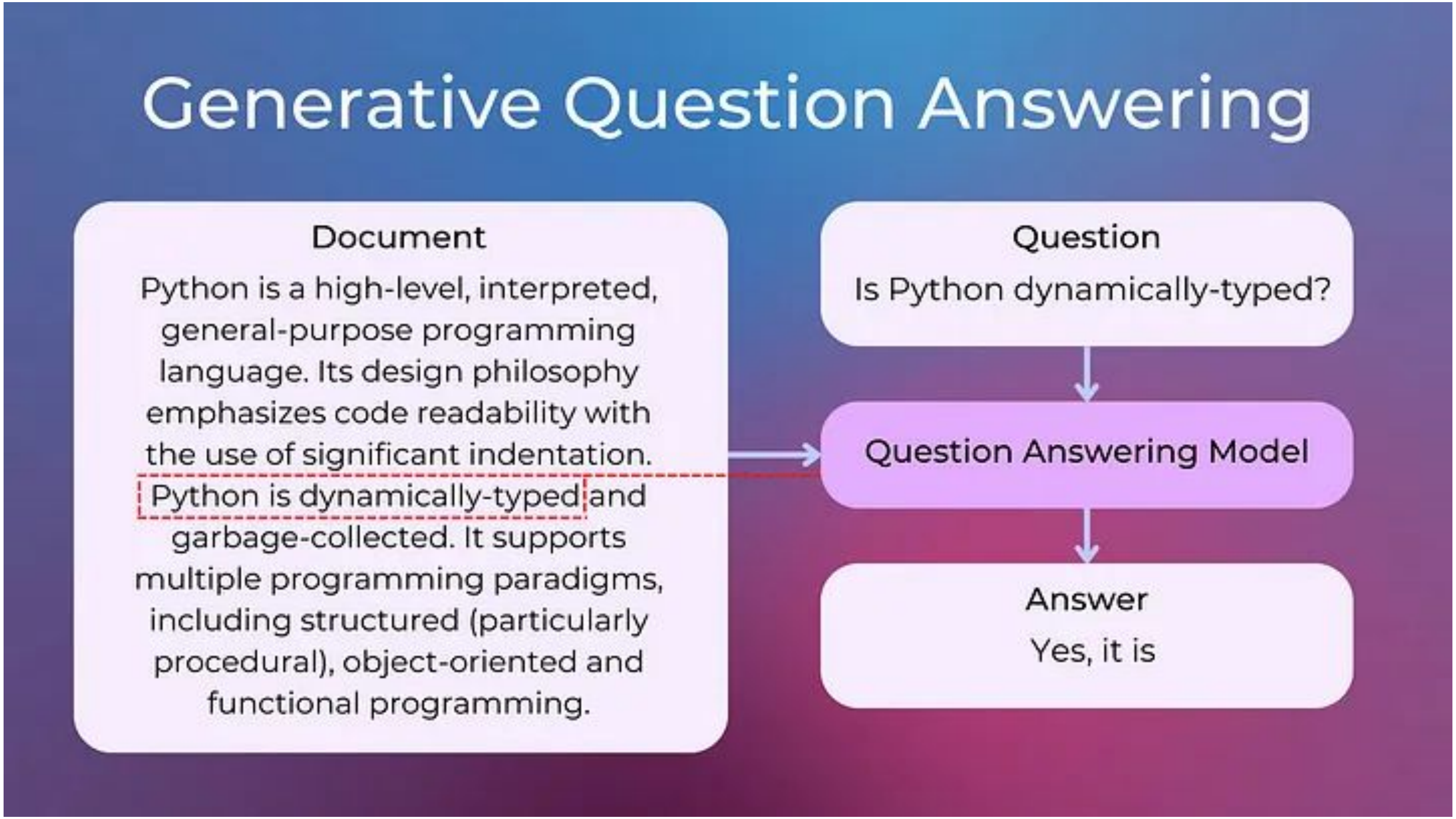
Overview of NLP tasks



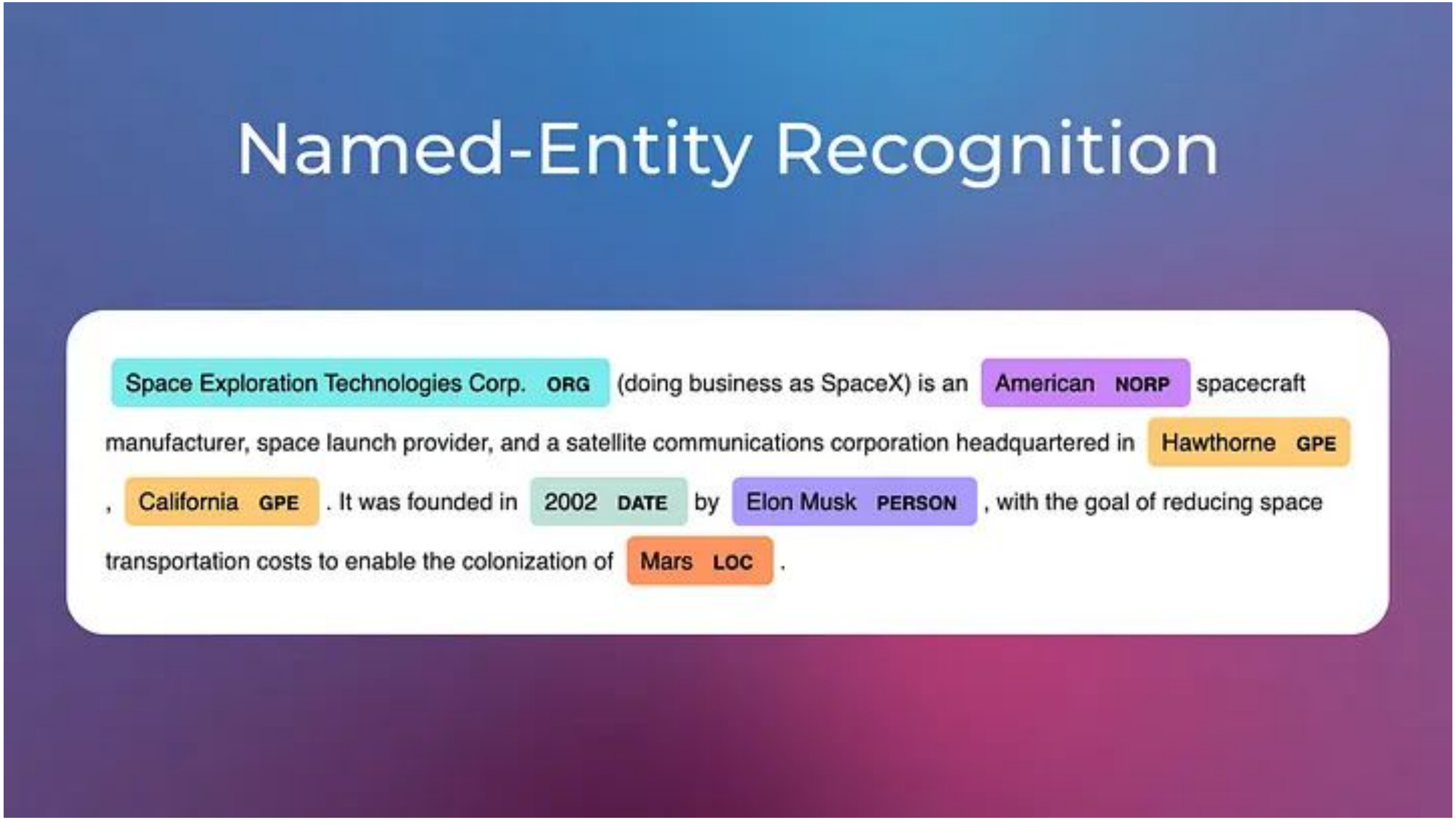
Overview of NLP tasks



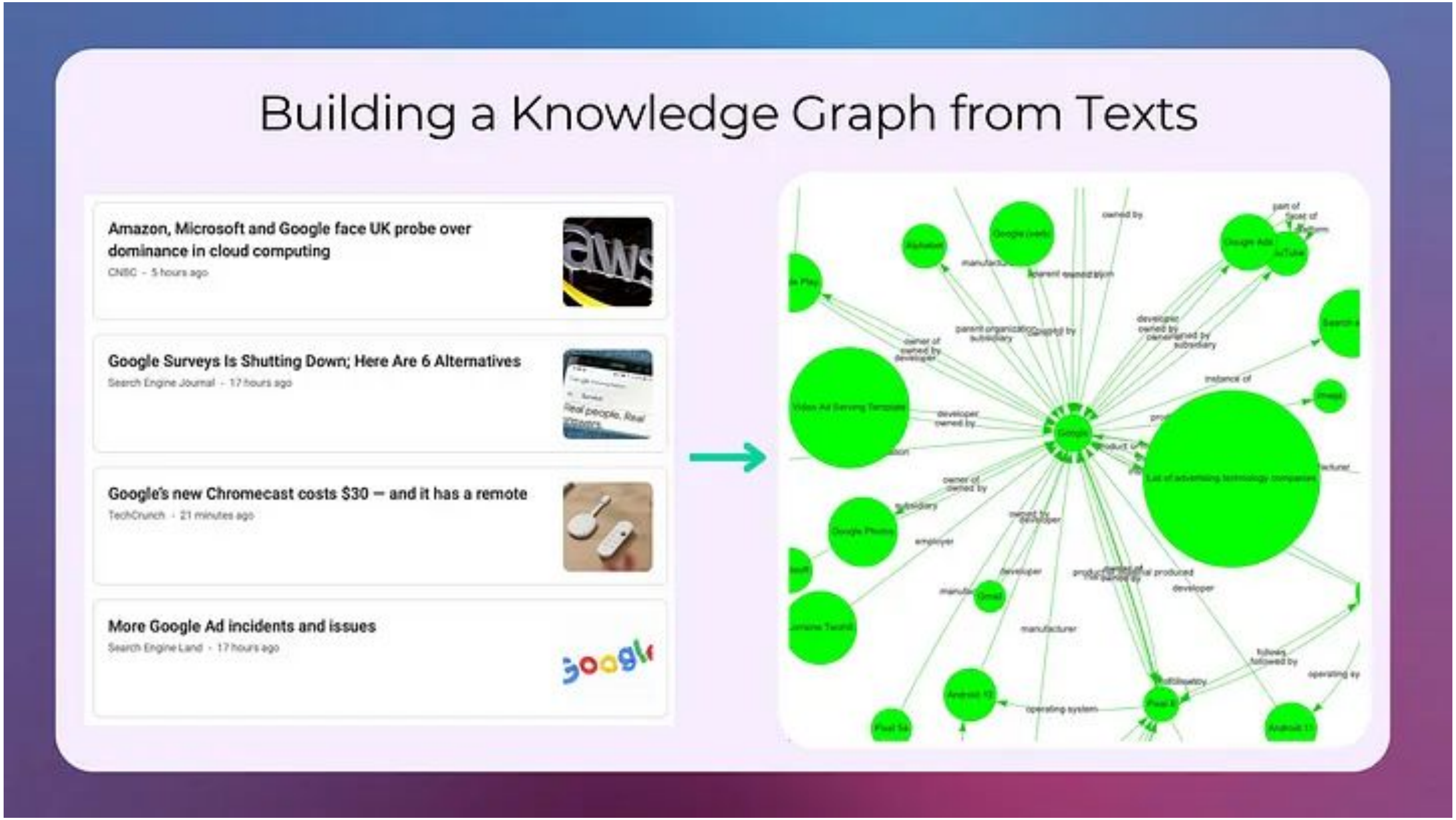
Overview of NLP tasks



Overview of NLP tasks

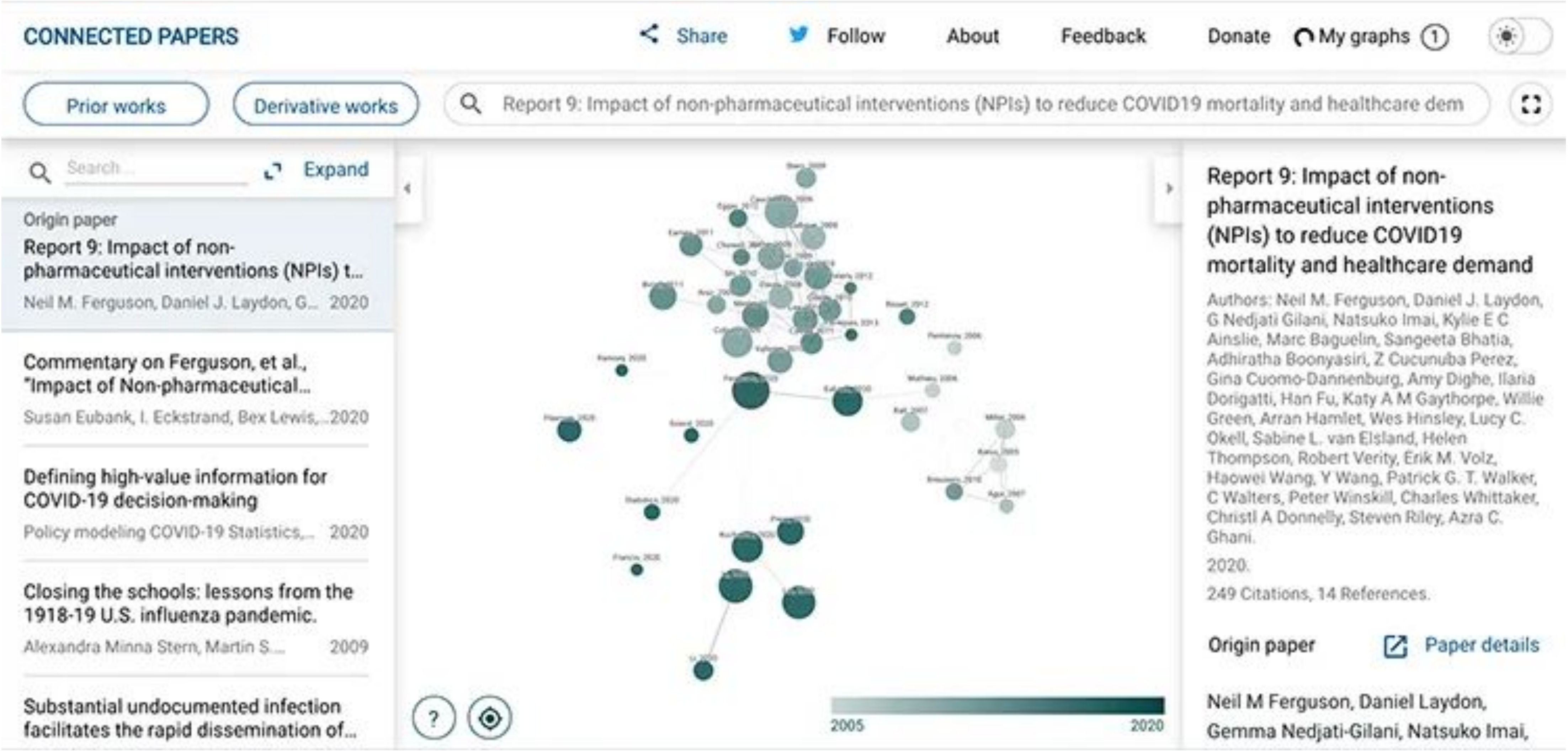


Overview of NLP tasks

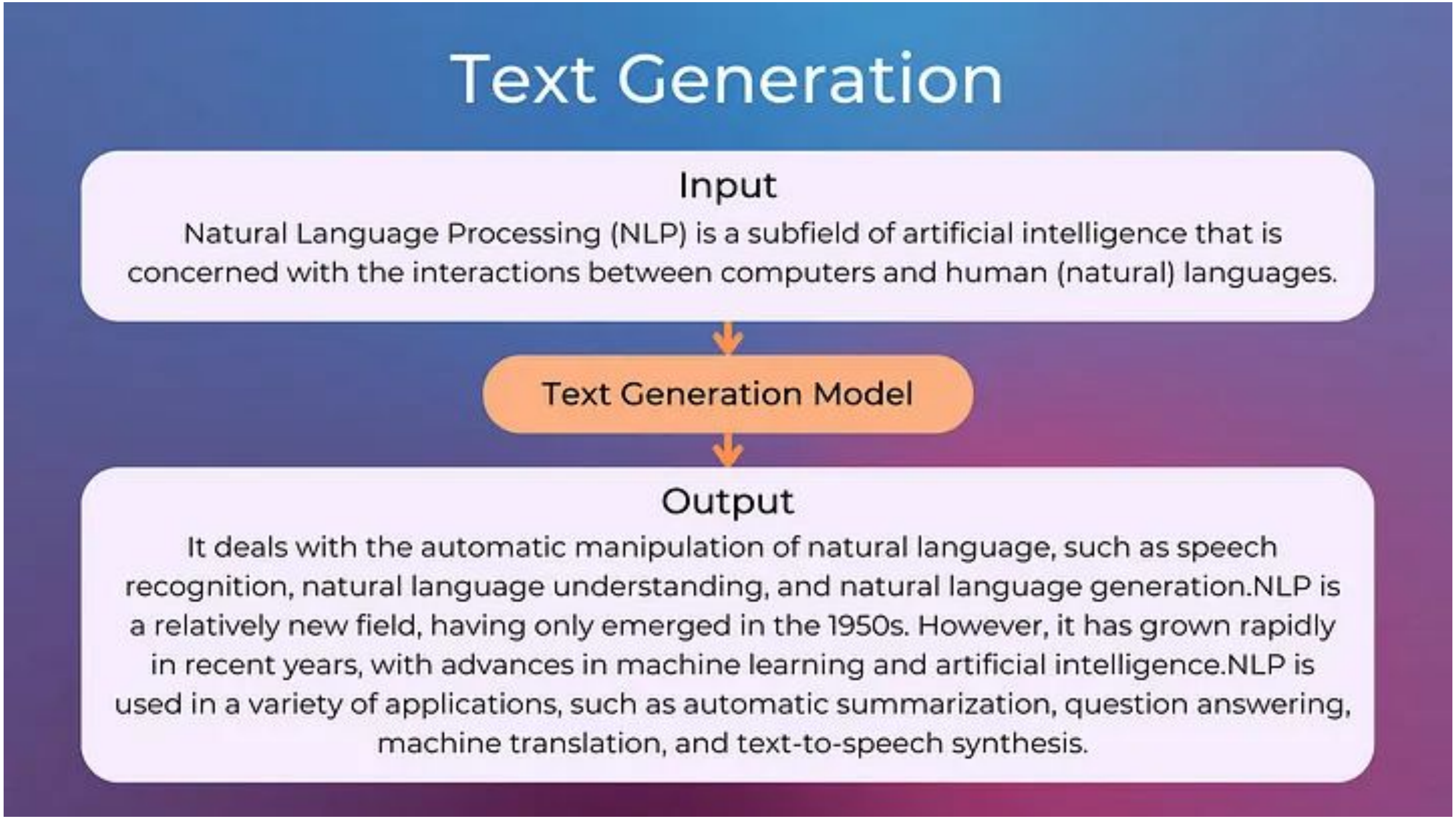


Overview of NLP tasks

Archive Connected Papers - NLP Knowledge Graph Based Innovation

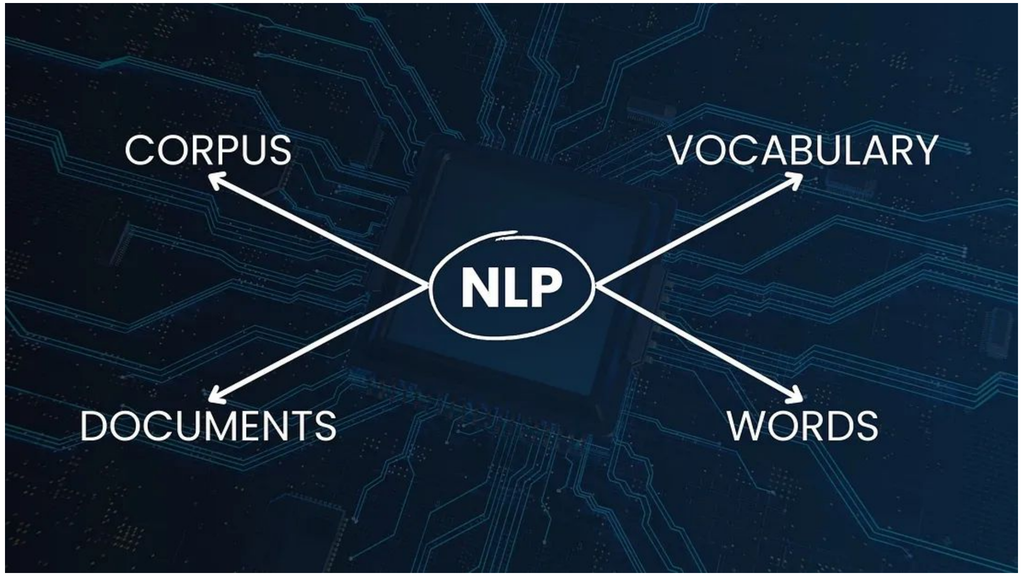


Overview of NLP tasks



Corpus , Documents , Vocabulary, Words

Corpus



- *Definition*: A large and structured set of texts, usually stored electronically used for linguistic research and NLP tasks.
- *Example*: A collection of all articles from Wikipedia.

Documents

- *Definition*: Individual pieces of text within a corpus.
- *Example*: Each Wikipedia article in the corpus.

Vocabulary

- *Definition*: The set of all unique words used across a corpus.
- *Example*: Words like "computer", "language", "processing" found in the Wikipedia corpus.

Words

- *Definition*: Independent lexical units; the smallest units of meaning in a language.
- *Example*: "Natural", "Language", "Processing", "NLP".

Tokenization

Tokenization is a process that splits an input sequence into so-called tokens

- You can think of a token as a useful unit for semantic processing
- Can be a word, sentence, paragraph, etc.

An example of simple whitespace tokenizer

- nltk.tokenize.WhitespaceTokenizer

This	is	Andrew's	text,	isn't	it?

- Problem: "it" and "it?" are different tokens with same meaning

Let's try to also split by punctuation

- nltk.tokenize.WordPunctTokenizer

This	is	Andrew	'	s	text	,	isn	'	t	it	?

- Problem: "s", "isn", "t" are not very meaningful

Example Tokenization Imp.

```
import nltk
text = "This is Andrew's text, isn't it?"
```

```
tokenizer = nltk.tokenize.WhitespaceTokenizer()
tokenizer.tokenize(text)
```

```
['This', 'is', "Andrew's", 'text,', "isn't", 'it?']
```

```
tokenizer = nltk.tokenize.WordPunctTokenizer()
tokenizer.tokenize(text)
```

```
['This', 'is', 'Andrew', "'", 's', 'text', ',', 'isn',
 "'", 't', 'it', '?']
```

Tokens Normalization

We may want the same token for different forms of the word

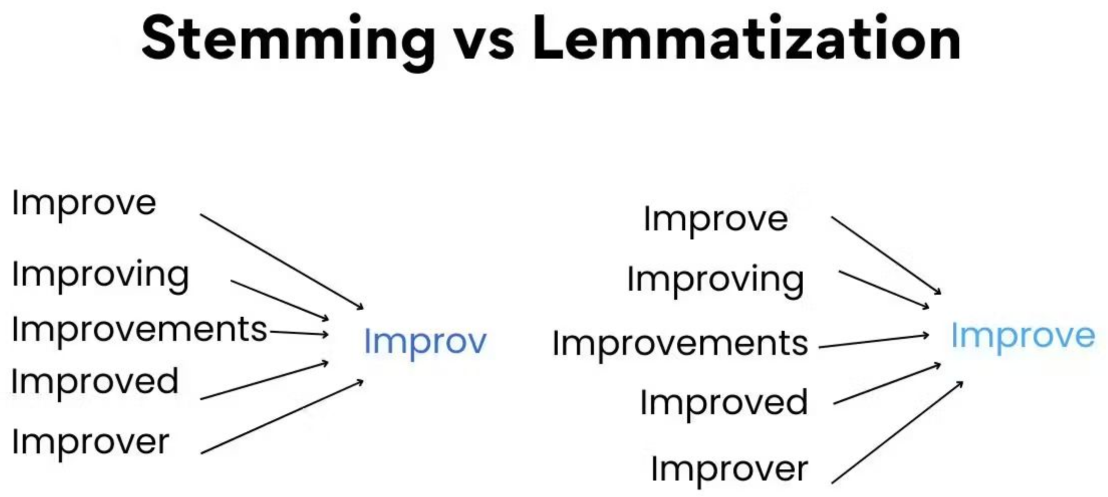
- wolf, wolves → wolf
- talk, talks → talk

Stemming

- A process of removing and replacing suffixes to get to the root form of the word, which is called the **stem**
- Usually refers to heuristics that chop off suffixes

Lemmatization

- Usually refers to doing things properly with the use of a vocabulary and morphological analysis
- Returns the base or dictionary form of a word, which is known as the **lemma**



Porter's stemmer

- 5 heuristic phases of word reductions, applied sequentially
- Example of phase 1 rules:

Rule	Example
SSES → SS	caresses → caress
IES → I	ponies → poni
SS → SS	caress → caress
S →	cats → cat

- nltk.stem.PorterStemmer
- Examples:
  - feet → feet　　cats → cat
  - wolves → wolv　　talked → talk
- Problem: fails on irregular forms, produces non-words

WordNet lemmatizer

- Uses the WordNet Database to lookup lemmas
- nltk.stem.WordNetLemmatizer
- Examples:
  - feet → foot   cats → cat
  - wolves → wolf   talked → talked
- Problems: not all forms are reduced

- Takeaway: we need to try stemming or lemmatization and choose best for our task

Tokens Normalization

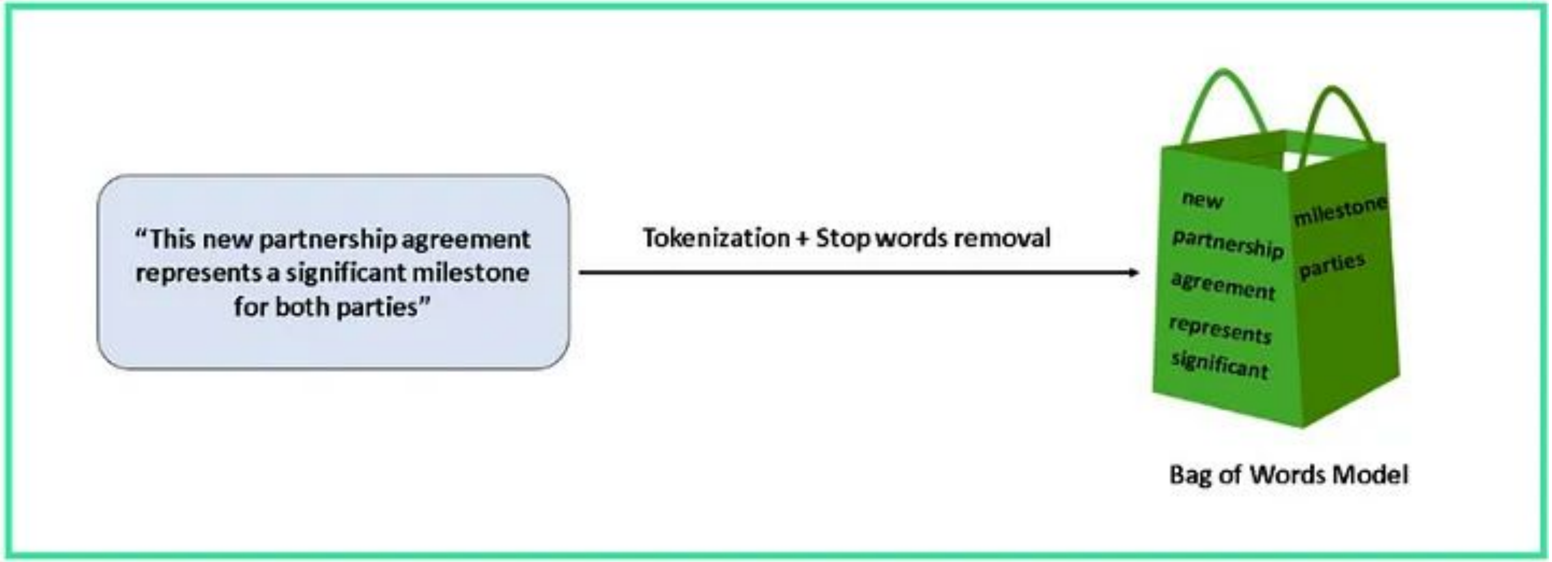
Normalizing capital letters

- Us, us → us (if both are pronoun)
- us, US (could be pronoun and country)
- We can use heuristics:
  - lowercasing the beginning of the sentence
  - lowercasing words in titles
  - leave mid-sentence words as they are

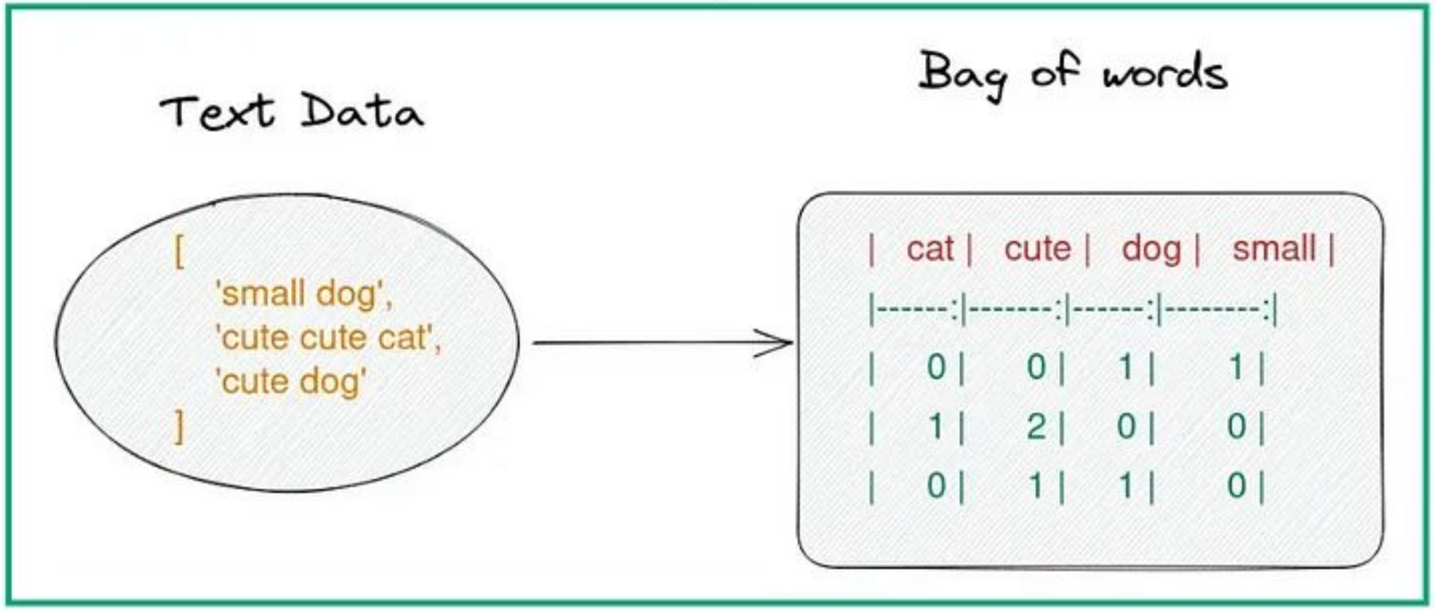
Acronyms

- eta, e.t.a., E.T.A. → E.T.A.
- We can write a bunch of regular expressions → hard

Token to features



Bag of words



Token to features

- Problems:
  - we loose word order, hence the name "bag of words"
  - counters are not normalized

Bag of words

Let's preserve some ordering

Solutions:

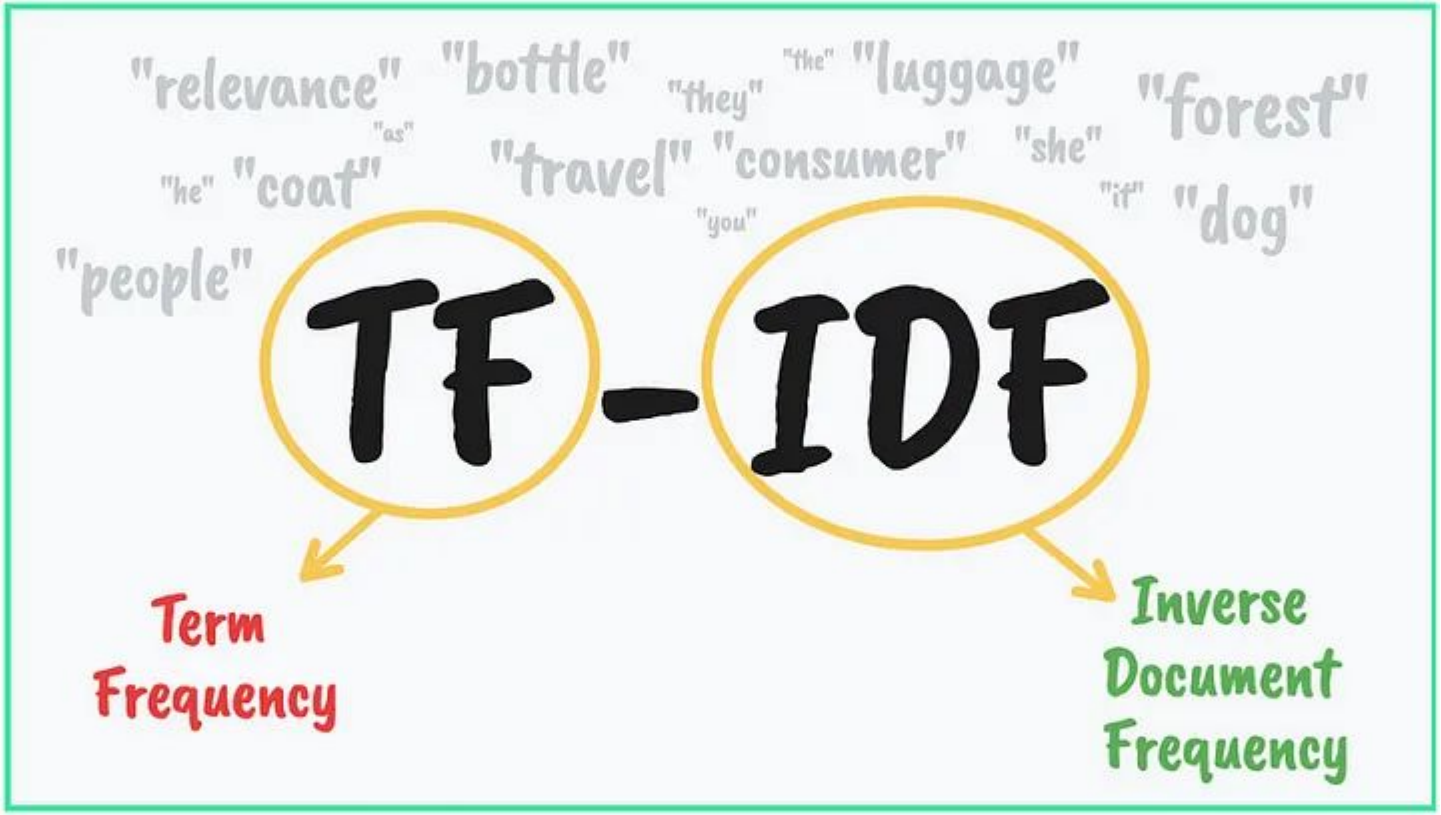
We can count token pairs, triplets, etc.

- Also known as n-grams
  - 1-grams for tokens
  - 2-grams for token pairs
  - ...

good movie
not a good movie
did not like

good movie	movie	did not	a	...
1	1	0	0	...
1	1	0	1	...
0	0	1	0	...

Token to features



$$TF(t, d) = \frac{\text{number of times t appears in d}}{\text{total number of words in d}}$$

$$IDF(t) = \log \frac{\text{Total number of documents}}{\text{number of documents that contain t}}$$

$$TF\text{-}IDF(t, d) = TF(t, d) \times IDF(t)$$